

Addressing Modes Of 8086

Introduction

```
;PROGRAM TO ADD TWO 16-BIT DATA (METHOD-1)

DATA SEGMENT                ;Assembler directive

    ORG 1104H                ;Assembler directive
    SUM DW 0                 ;Assembler directive
    CARRY DB 0               ;Assembler directive

DATA ENDS                    ;Assembler directive

CODE SEGMENT                 ;Assembler directive

    ASSUME CS:CODE           ;Assembler directive
    ASSUME DS:DATA           ;Assembler directive
    ORG 1000H                ;Assembler directive

    MOV AX,205AH             ;Load the first data in AX register
    MOV BX,40EDH             ;Load the second data in BX register
    MOV CL,00H               ;Clear the CL register for carry
    ADD AX,BX                 ;Add the two data, sum will be in AX
    MOV SUM,AX               ;Store the sum in memory location (1104H)
    JNC AHEAD                 ;Check the status of carry flag
    INC CL                    ;If carry flag is set,increment CL by one
AHEAD: MOV CARRY,CL           ;Store the carry in memory location (1106H)
    HLT

CODE ENDS                    ;Assembler directive
END                          ;Assembler directive
```

Program

A set of instructions written to solve a problem.

Instruction

Directions which a microprocessor follows to execute a task or part of a task.

Computer language

High Level

Low Level

Machine Language

Assembly Language

- Binary bits

- English Alphabets
- 'Mnemonics'
- Assembler Mnemonics → Machine Language

Introduction... Cont'd

Program is a set of instructions written to solve a problem. Instructions are the directions which a microprocessor follows to execute a task or part of a task. Broadly, computer languages can be divided into two parts as high-level language and low level language. Low level languages are machine specific. Low level language can be further divided into machine language and assembly language.

Machine language is the only language which a machine can understand. Instructions in this language are written in binary bits as a specific bit pattern. The computer interprets this bit pattern as an instruction to perform a particular task. The entire program is a sequence of binary numbers. This is a machine-friendly language but not user friendly. Debugging is another problem associated with machine language.

To overcome these problems, programmers develop another way in which instructions are written in English alphabets. This new language is known as **Assembly** language. The instructions in this language are termed **mnemonics**. As microprocessor can only understand the machine language so mnemonics are translated into machine language either manually or by a program known as **assembler**.

Efficient software development for the microprocessor requires a complete familiarity with the instruction set, their format and addressing modes. Here in this chapter, we will focus on the addressing modes and instructions formats of microprocessor 8086.

ADDRESSING MODES OF 8086

Data Addressing Modes

- Every Instruction of a program has to operate on a data/operand.
- Different ways in which source operand is denoted in an instruction are known as addressing modes.
- Addressing mode is the way in which the operand is specified.
- The different ways in which a processor can access data are called addressing modes.
- Addressing mode indicates a way of locating data or operands.

Various Data Addressing Modes

- 1. Immediate Addressing**
- 2. Register Addressing**
- 3. Direct Addressing**
- 4. Register Indirect Addressing**
- 5. Based Addressing**
- 6. Indexed Addressing**
- 7. Based Indexed Addressing**
- 8. String Addressing**
- 9. Port Addressing**

1. IMMEDIATE ADDRESSING MODE

- In immediate addressing mode, an 8-bit or 16-bit immediate data is directly specified as part of the instruction.
 - **Example 1:** `MOV DL, 008H` $(DL) \leftarrow 08_H$
- The 8-bit data (08_H) given in the instruction is moved to DL. The immediate value has to start with a '0'. This '0' has no meaning at all. It is a reminder for the programmer that the instruction is in immediate mode
 - **Example 2:** `MOV AX, 00A9FH`
- The 16-bit data ($0A9F_H$) given in the instruction is moved to AX register.



- In this addressing mode, the **segment register** can't be **destination operand**.

2.REGISTER ADDRESSING MODE

- The instruction will specify the name of the register which holds the data to be operated by the instruction.

- Source & Destination Operands: Registers

- **Example 1: 8 bit register as source of operand**

MOV CL, DH

- The content of 8-bit register DH is moved to another 8-bit register CL
 $(CL) \leftarrow (DH)$

- **Example 2: 16 bit register as source of operand**

MOV AX, BX

3. DIRECT ADDRESSING MODE

- Here, the Effective address of the memory location at which the data operand is stored is specified in the instruction.

Example: `MOV BX, [1234H]` or `MOV BX, 1234H` or `MOV BX, Name`

- Please note that Name is a predefined value and is equal to 1234H
- Here if DS=2000H and offset (Effective Address) =1234H
Physical Address=DS*10H+Offset Address
 $=2000H*10H+1234H =21234H$
- This instruction moves the contents of 21234H & 21235H memory location into BL and BH register respectively.
- The **square brackets** around the 1234_H denotes the **contents of the memory location**. When executed, this instruction **will copy the contents of the memory locations; 1234_H & 1235_H, in the data segment into BX register**.
- This addressing mode is called **direct** because the **displacement of the operand from the segment base** is specified directly in the instruction.

4. REGISTER INDIRECT ADDRESSING MODE

- In Register indirect addressing, name of the register which holds the **Effective Address (EA)** will be specified in the instruction.
- Registers used to hold **Effective Address of an operand** are any of the following registers: **BX, SI, DI and BP**.

- **Example:** MOV AX, [BX]

Operations:

EA = (BX)
BA = (DS) x 10 H
MA = BA + EA

(AX) ← (MA) or,

(AL) ← (MA)
(AH) ← (MA + 1)

Note : Register/ memory enclosed in brackets refer to content of register/ memory

where:

DS = Data Segment

EA = Effective Address

BA = Base Address

MA = (Physical) Memory Address

5. INDEXED ADDRESSING MODE

- **SI or DI** register is used to hold an index value for memory data and a signed 8-bit or unsigned 16-bit displacement will be specified in the instruction.
- Displacement is added to the index value in SI or DI register to obtain the **Effective Address (EA)**.
- **In case of 8-bit displacement**, it is **sign extended** to 16-bit before adding to the base value.

- **Example:** MOV CX, DISP[SI] ; indexing addressing mode with displacement
where DISP=A2H

- **Operations:**

$FFA2_H \leftarrow A2_H$ (Sign extended)

$EA = (SI) + FFA2_H$

$BA = (DS) \times 10\ H$

$MA = BA + EA$

$(CX) \leftarrow (MA)$ or,

$(CL) \leftarrow (MA)$

$(CH) \leftarrow (MA + 1)$

6. BASED ADDRESSING MODE

- In Based Addressing, **BX** or **BP** is used to hold the base value for effective address and a **signed 8-bit or unsigned 16-bit** displacement will be specified in the instruction.
- In case of 8-bit displacement, it is **sign extended** to 16-bit before adding to the base value.
- When **BX** holds the base value of **Effective Address (EA)**, 20-bit physical address is calculated from **BX** and **DS**.
- When **BP** holds the base value of EA, **BP** and **SS** is used.
 - **Example:** `MOV AX, [BX].DISP`; Based Addressing Mode with displacement where `DISP=08H`

Operations:

$0008_H \leftarrow 08_H$ (Sign extended)

$EA = (BX) + 0008_H$

$BA = (DS) \times 10_H$

$MA = BA + EA$

$(AX) \leftarrow (MA)$ or,

$(AL) \leftarrow (MA)$

$(AH) \leftarrow (MA + 1)$

7.BASED INDEX ADDRESSING MODE

- In Based Index Addressing, the effective address is computed from the sum of a base register (BX or BP), an index register (SI or DI) and a displacement.

• **Example** **MOV DX, [BX].DISP[SI]** assume DISP=0AH

Operations:

$000A_H \leftarrow 0A_H$ (Sign extended) EA

$= (BX) + (SI) + 000A_H$

$BA = (DS) \times 10 H$

$MA = BA + EA$

$(DX) \leftarrow (MA)$ or,

$(DL) \leftarrow (MA)$

$(DH) \leftarrow (MA + 1)$

8. STRING ADDRESSING MODE

- Employed in string operations to operate on string data.
- The effective address (EA) of source data is stored in **SI register** and the **EA** of destination is stored in **DI register**.
- **Segment register** for calculating base address of source data is **DS** and that of the destination data is **ES**.

- **Example:** MOVSB ;

Operations:

Calculation of source memory location:

$$EA_D = (SI) \quad BA_D = (DS) \times 10H \quad MA_D = BA_D + EA_D$$

Calculation of destination memory location:

$$EA_E = (DI) \quad BA_E = (ES) \times 10H \quad MA_E = BA_E + EA_E$$

Note : Effective address of the Data Segment

Note : Effective address of the Extra segment

$$(MA_E) \leftarrow (MA_D)$$

If $DF = 1$, then $(SI) \leftarrow (SI) - 1$ and $(DI) \leftarrow (DI) - 1$

If $DF = 0$, then $(SI) \leftarrow (SI) + 1$ and $(DI) \leftarrow (DI) + 1$

9. PORT ADDRESSING MODE

- These addressing modes are used to access data from standard I/O mapped devices or ports.
- Standard I/O devices use port addressing modes
- In **direct port addressing mode**, an 8-bit port address is directly specified in the instruction.

• **Example:** `IN AL, 09H`

Operations:

$\text{PORT}_{\text{addr}} = 09_{\text{H}}$
 $(\text{AL}) \leftarrow (\text{PORT})$

Content of port with address 09_{H} is moved to AL register

INSTRUCTION SET of 8086

Instruction set basics

- **Instruction:-** An instruction is a **binary pattern** designed inside a microprocessor to perform a **specific function**.
- **Opcode:-** It stands for operational code. It specifies the **type of operation to be performed by CPU**. It is the first field in the machine language instruction format.
- E.g. 08 is the opcode for instruction “MOV X,Y”.
- **Operand:-** We can also say it as **data on which operation should act**. operands may be register values or memory values. The CPU executes the instructions using information present in this field. It may be 8-bit data or 16-bit data.

Instruction set basics

- **Assembler:-** it converts the instruction into sequence of binary bits, so that this bits can be read by the processor.
- **Mnemonics:-** these are the **symbolic codes** for either instructions or commands to perform a particular function.
- **E.g. MOV, ADD, SUB etc.**

Types of instruction set of 8086 microprocessor

1. Data Transfer/Copy Instructions
2. Arithmetic Instructions
3. Bit Manipulation Instructions
 - I. Logical Instructions
 - II. Shift & Rotate Instructions
4. Control Transfer (Branch) Instructions
5. Iteration Control (Loop) Instructions
6. Flag Manipulation & Machine (Processor) Control Instructions
 - I. Flag Manipulation Instructions
 - II. Processor Control Instructions
7. String Instructions
8. Interrupt Instructions

(1). Data copy/transfer instructions.

- Instructions that are used to copy/transfer data/ address in to registers, memory locations
- Generally involve two operands: Source operand and Destination operand of the same size.
- **Source**: Register or a memory location or an immediate data
- **Destination** : Register or a memory location.
- The size should be a either a byte or a word.
- A 8-bit data can only be moved to 8-bit register/ memory and a 16-bit data can be moved to 16-bit register/ memory.

MOV instruction

Mnemonic	Meaning	Format	Operation	Flags Affected
MOV	Move	MOV D,S	(S) → (D)	None

- It is used to transfer a byte or a Word of data from a source operand to a destination operand
- MOV instruction cannot transfer data directly between a source and destination that both reside in external memory.

MOV instruction

- MOV reg/mem MOV BX, 0210H
 - MOV reg /reg MOV AL, BL
 - MOV mem/Imm MOV [DI], 01231H
 - MOV reg /Imm MOV AL, 011H
 - MOV Acc. /mem MOV AL, [SI]
 - MOV Seg /mem MOV SS, [DI]
 - MOV reg / Seg MOV DX, SS
-
- MOV CS,DS ?
 - MOV BL, BX ?
 - MOV [SI], [BX] ?

Destination	Source
Memory	Accumulator
Accumulator	Memory
Register	Register
Register	Memory
Memory	Register
Register	Immediate
Memory	Immediate
Seg-reg	Reg16
Seg-reg	Mem16
Reg16	Seg-reg
Mem16	Seg-reg

MOV instruction

In the case of immediate addressing mode, a segment register cannot be destination register. In other words, direct loading of the segment registers with immediate data is not permitted. To load the segment registers with immediate data, one will have to load any general-purpose register with the data and then it will have to be moved to that particular segment register.

Ex: Load DS with 5000H

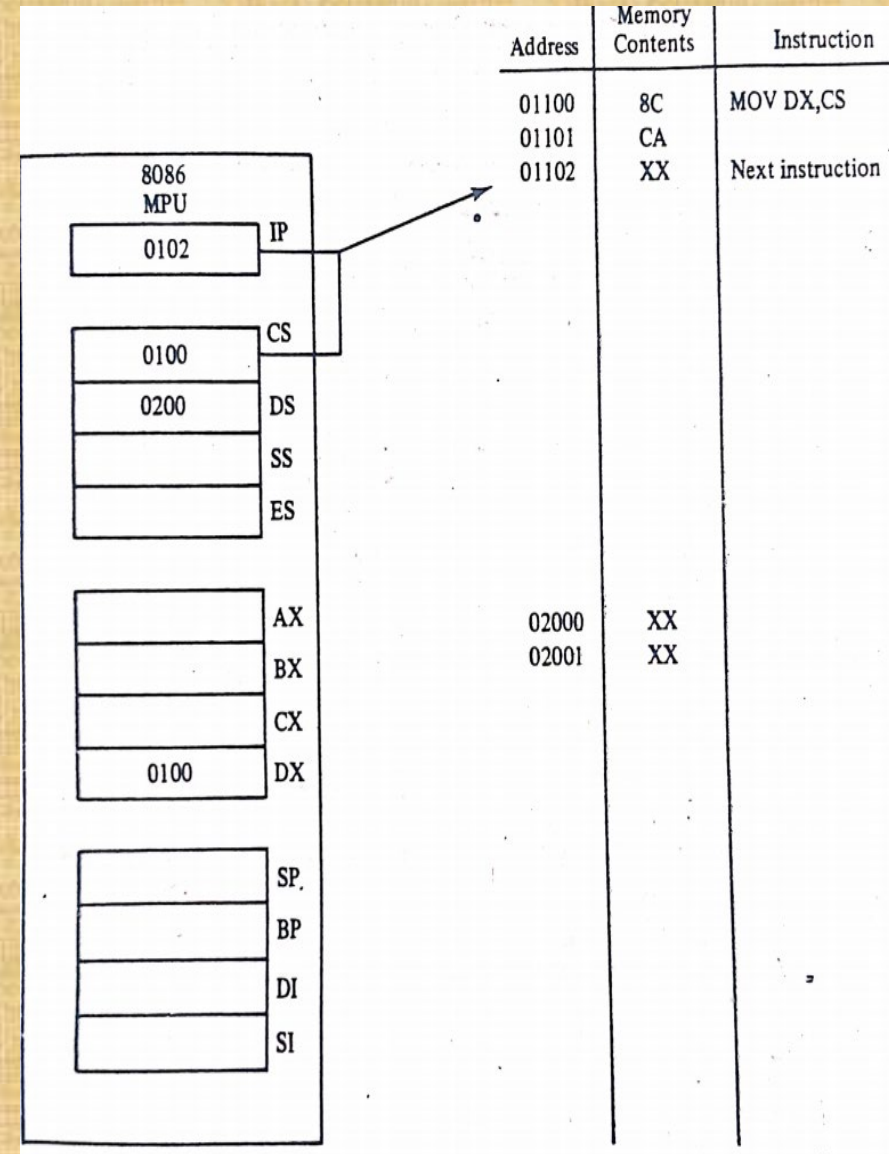
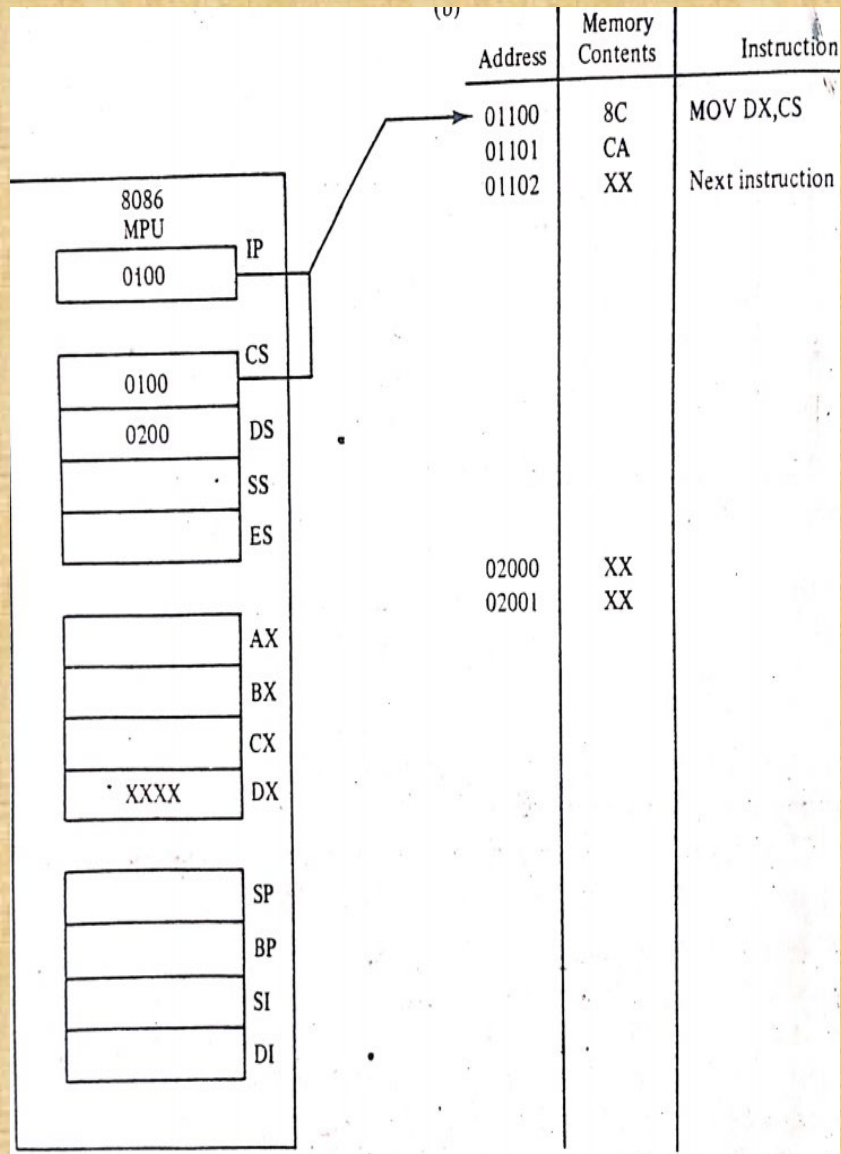
MOV DS, 05000H; Not permitted (invalid)

Thus to transfer an immediate data into the segment register, the convert procedure is given below:

MOV AX, 05000H
MOV DS, AX

Both the source and destination operands cannot be memory locations (Except for string instructions)

MOV instruction



XCHG instruction

Mnemonic	Meaning	Format	Operation	Flags Affected
XCHG	Exchange	XCHG D,S	(D) $\leftrightarrow$ (S)	None

Destination	Source
Accumulator Memory Register	Reg16 Register Register

- This instruction exchanges the contents of the specified source and destination operands, which may be registers or one of them may be a memory location. However, exchange of data contents of segment registers or two memory locations are **not permitted**.

Ex: XCHG Acc, Reg16
XCHG Reg, Reg

XCHG AX, DX
XCHG AL, CL

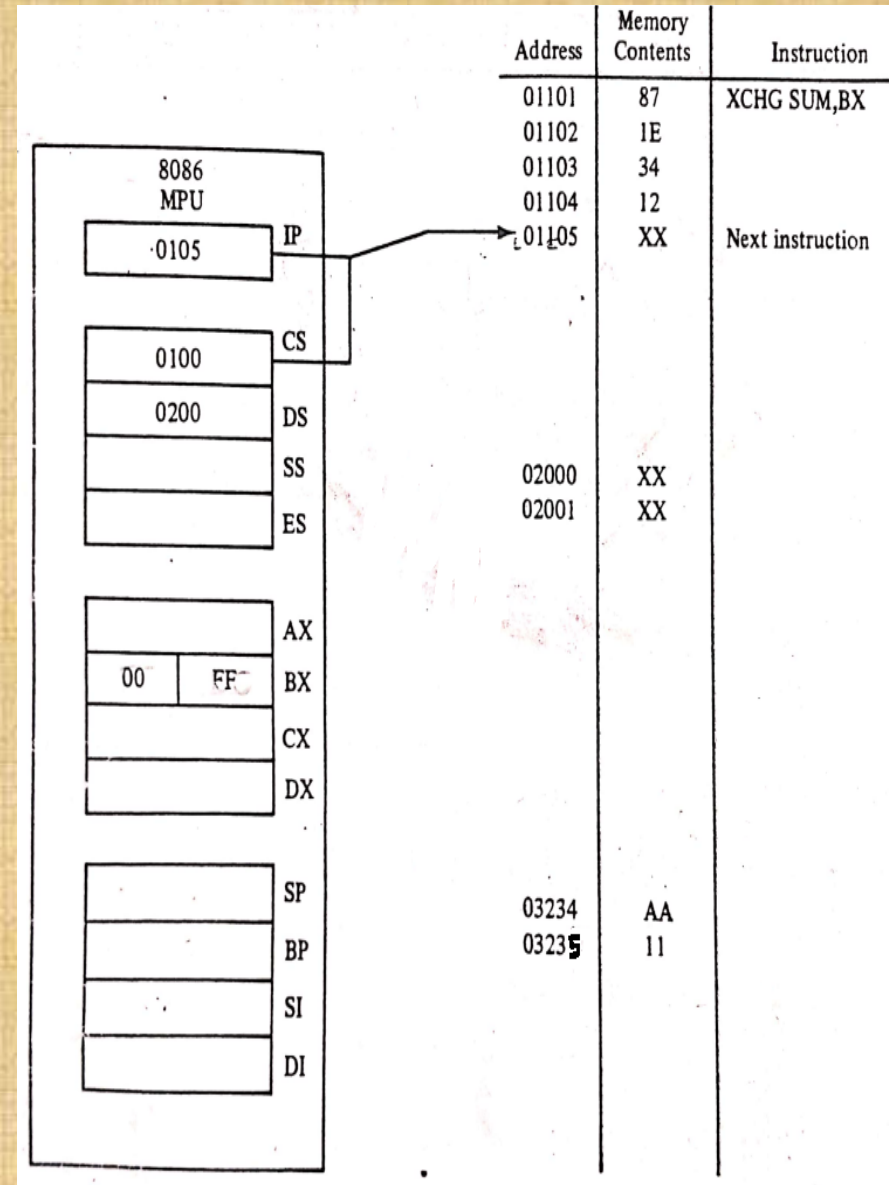
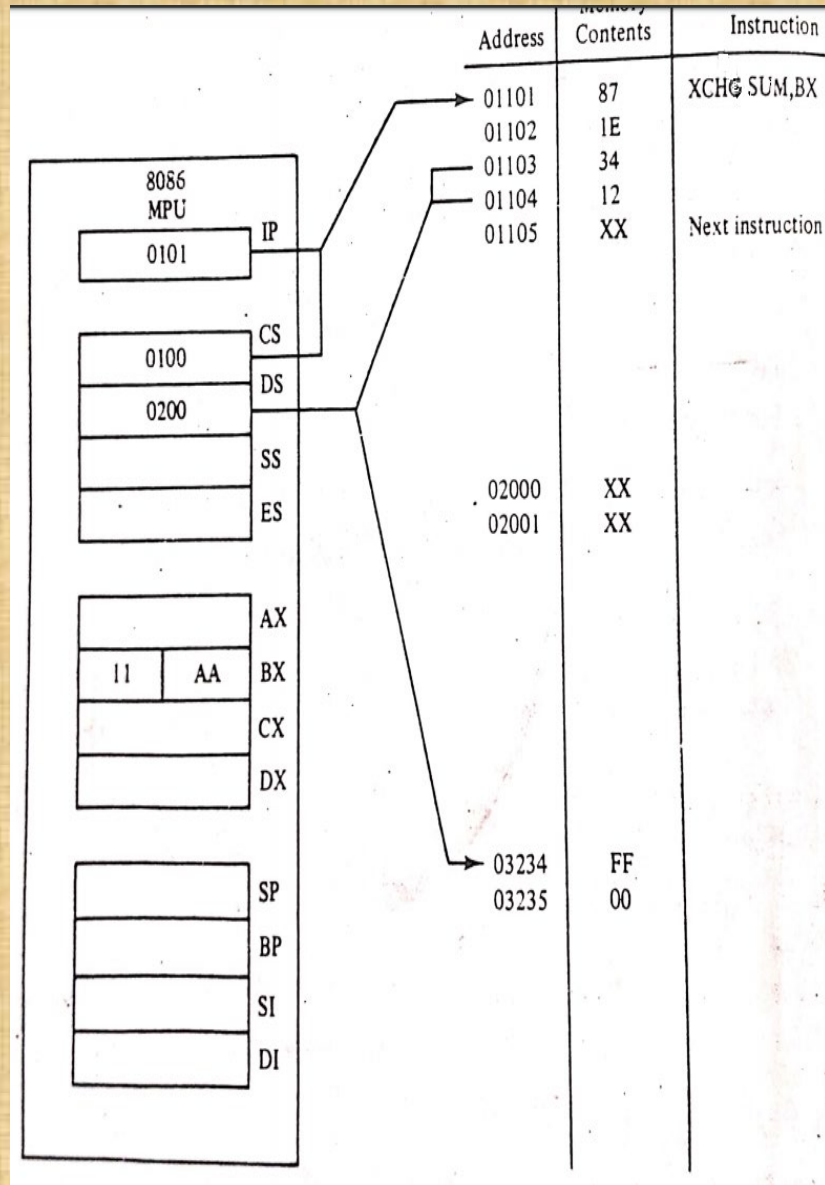
Ex: XCHG Mem , Reg
XCHG Reg,Mem

XCHG [BX], DX
XCHG AL, SUM[BX] ???

Notation?

XCHG AL, SUM[DI or SI]
XCHG AL, [BX].SUM

XCHG instruction



XLAT instruction

Mnemonic	Meaning	Format	Operation	Flags Affected
XLAT	Translate	XLAT Source-table	$((AL) + (BX) + (DS)0) \rightarrow (AL)$	None

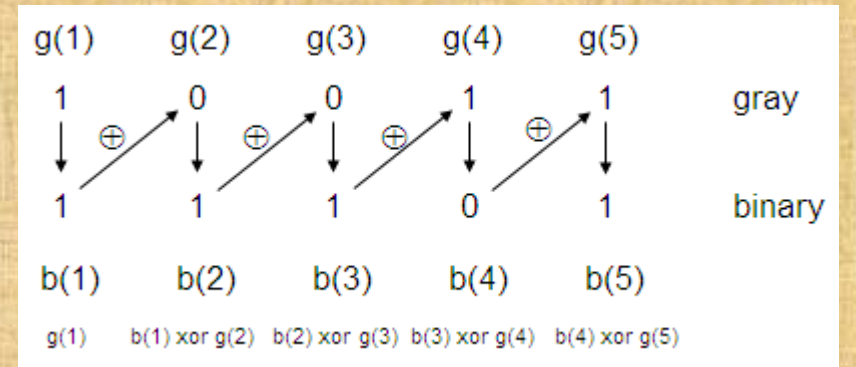
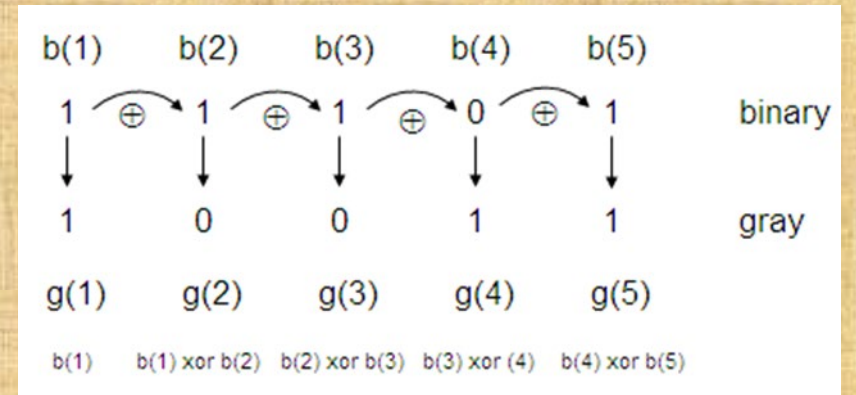
- Also known as **translate** instruction.
- Used to translate a byte in AL using a table in the memory.
- E.g. XLAT; Translate a byte in AL
 - This instruction replaces a byte in AL register with a byte from a lookup table in memory
 - BX register stores the offset of the starting address of the lookup table and AL register stores the byte number from the lookup table.
 - This instruction copies byte from address pointed by $[(DS)0 + BX + AL]$ back into AL.

XLAT instruction

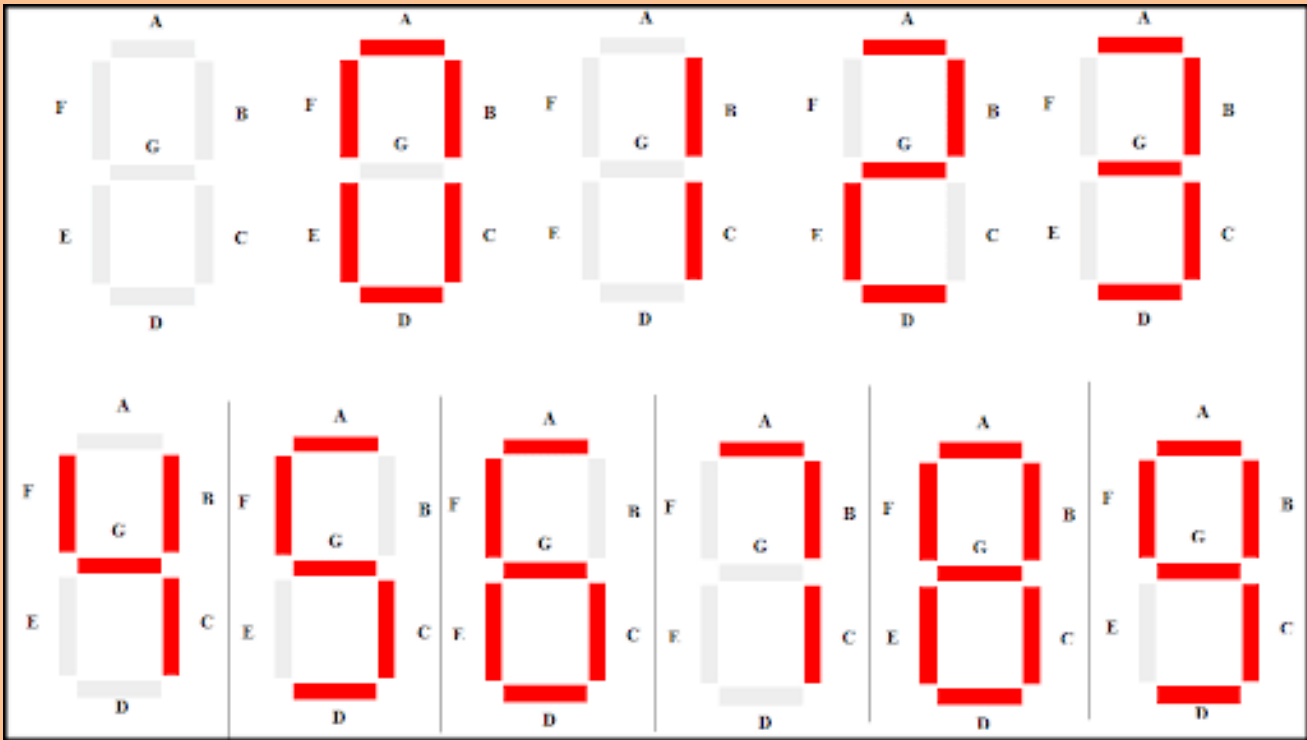
It is used to find out codes in case of code conversion.

- i.e. it translates code of the key pressed to the corresponding 7-segment code.
- After execution this instruction contents of AL register always gets replaced.

Decimal	Binary	Gray	BCD	Excess-3
0	0	0000	0000	0011
1	1	0001	0001	0100
2	10	0011	0010	0101
3	11	0010	0011	0110
4	100	0110	0100	0111
5	101	0111	0101	1000
6	110	0101	0110	1001
7	111	0100	0111	1010
8	1000	1100	1000	1011
9	1001	1101	1001	1100



Address	Memory	Address	Memory	Address	Memory
10100	00	10200	03	10300	3F
10101	01	10201	04	10301	06
10102	03	10202	05	10302	5B
10103	02	10203	06	10303	4F
10104	06	10204	07	10304	66
10105	07	10205	08	10305	6D
10106	05	10206	09	10306	7D
10107	04	10207	0A	10307	07
10108	0C	10208	0B	10308	7F
10109	0D	10209	0C	10309	6F
1010A	0F			1030A	
1010B				1030B	



0	G	F	E	D	C	B	A
0	0	1	1	1	1	1	1
0	0	0	0	0	1	1	0
0	1	1	0	1	1	0	1
0	1	1	1	1	1	1	1

3F	0
06	1
6D	5
7F	8

XLAT instruction

	AH	AL	DS	DI	BX	MEMORY	
MOV AX,01000H							
MOV DS,AX							
MOV DI,02000H							
MOV BX,00100H							
MOV AL,007H							
XLAT							
MOV [DI], AL							
INC DI							
MOV BX,00300H							
MOV AL,007H							
XLAT							
MOV [DI], AL							
INC DI							
MOV BX,00200H							
MOV AL,007H						12000	
XLAT						12001	
MOV [DI], AL						12002	

[illegible]

LEA, LDS and LES instructions

Mnemonic	Meaning	Format	Operation	Flags Affected
LEA	Load effective address	LEA Reg 16,EA	EA → (Reg 16)	None
LDS	Load register and DS	LDS Reg16,Mem32	(Mem32) → (Reg16) (Mem32+2) → (DS)	None
LES	Load register and ES	LES Reg16,Mem32	(Mem32) → (Reg16) (Mem32+2) → (ES)	None

- **LEA: Load Effective Address**
Used to **load the address of operand** into the provided register

LEA Reg, Source

This instruction determines the offset of the variable or memory location named as '**Source**' and load this address in the specified 16-bit register.

LEA instruction

MOV SI, 1234H

The instruction MOV reg,addr means read a variable stored at address addr into register reg.

LEA SI, 1234H

The instruction LEA reg,addr means read the address (not the variable stored at the address) into register reg.

MOV SI, 01234H

LEA SI, 1234H

LEA instruction is essentially the same as MOV reg, imm

MOV SI, SUM

LEA SI, SUM

where SUM=1234H

Try to load SUM address to SI register by using MOV instruction???

(9). LDS 16-bit register (source), address(dest.);

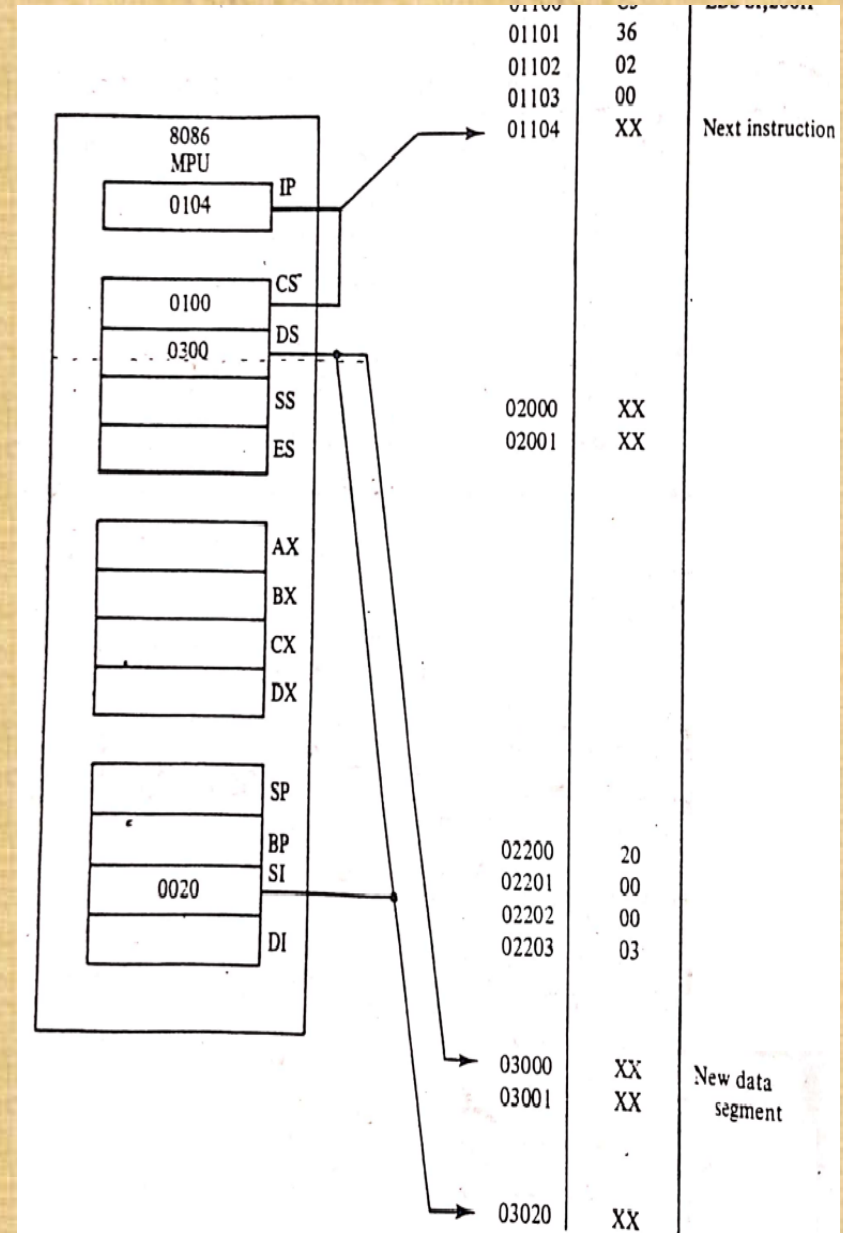
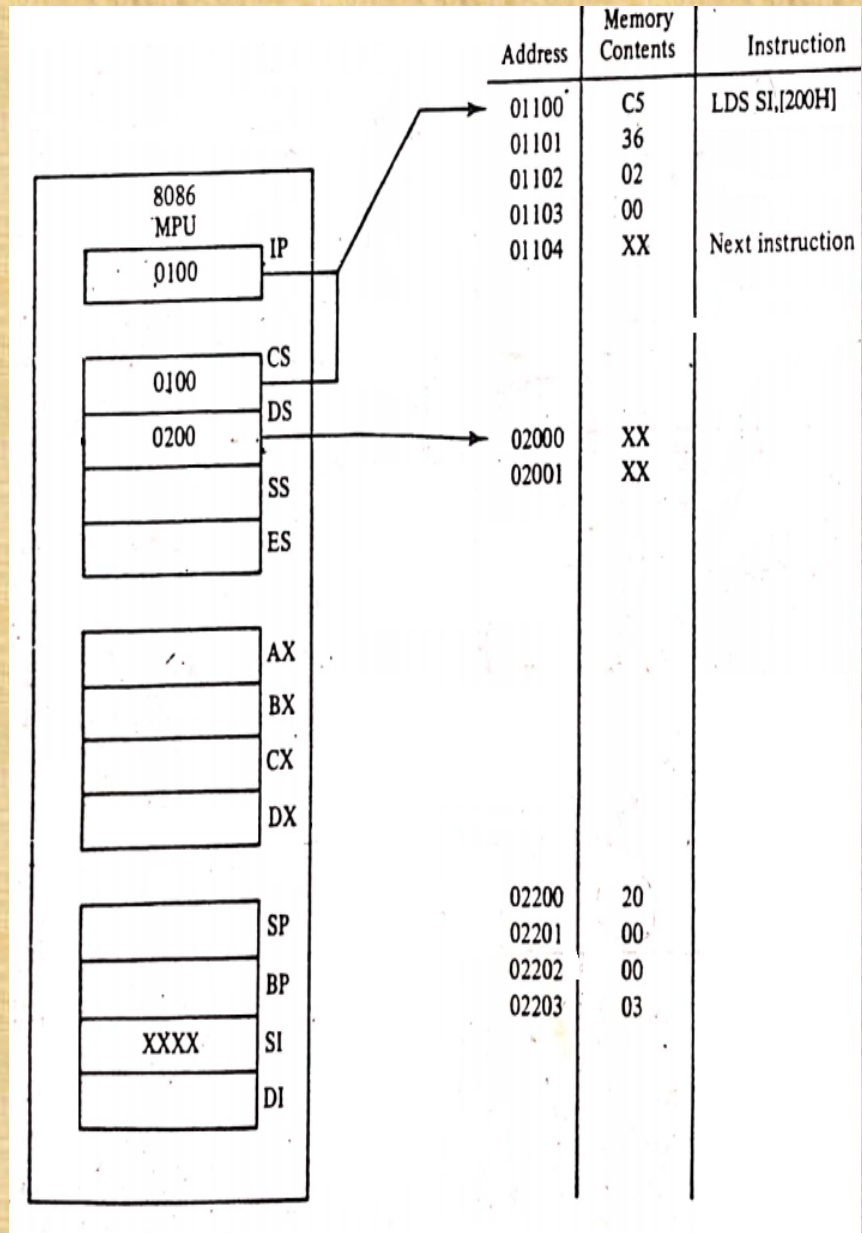
- LDS Also known as Load Data Segment (LDS).
- Used to load DS register and other provided register with words from memory.
- **LDS Reg,Mem**
 - This instruction loads new values into the specified register, **Reg**, and into the **DS** register from **four successive memory locations**.
 - This instruction copies words from two memory locations into the register, **Reg**, specified in the instruction.
 - It then copies a word from the next two memory locations into the **DS** register.

• E.g. `LDS CX, [391AH]` ; Copy contents of memory at displacement of
; 391AH and 391BH to CX. Then copy contents at
; displacement of 391CH and 391DH in DS.

(10). LES 16-bit register (source), address(dest.);

- LES Also known as Load Extra Segment (LES).
- Used to load ES register and other provided register with words from memory.
- **LES REG, Mem**
 - This instruction loads new values into the specified register, **REG**, and into the **ES** register from **four successive memory locations**.
 - The word from the first two memory locations is copied into the specified register, **REG**, specified in the instruction.
 - It then copies a word from the next two memory location is copied into **ES** register.

• E.g. `LES CX, [3483H]` ; Copy contents of memory at displacement of 3483H0
; in DS to CL, contents of 3484H in DS to CH and
; copy the contents of memory at displacement of
; 3485H and 3486H in DS to ES register.



See You Next Week!

STAY HOME

STAY HEALTHY

SAVE LIVES